

Hash cash POW GUI 버전

1. 해시 캐시 문제는 임의의 메시지에 nonce 숫자를 붙여서 해시를 시킨 결과가 난이도의 개수 만큼 0을 갖는 해시 출력을 갖도록 하는 nonce를 찾는 문제이다. 예를 들어 메시지가 Pyeongtaek University이고 난이도를 5라 하면 Pyeongtaek Univeristy + nonce를 해시 시켰을 때 00000 (다섯 개의 0) 으로 시작하는 해시값을 갖도록 하는 nonce를 찾는 문제이다. 아래는 이러한 Hash cash 문제를 구할 수 있도록 GUI 버전으로 구현한 것이다. 메시지 결과화면을 보면 Pyeongtaek University 메시지에 대해 00000으로 시작하는 해시값을 갖는 nonce는 1673654임을 알 수 있다. 그렇다면, 과연 난이도 10에 대해서는 nonce가 무엇일까? 어떠한 전략을 사용하면 쉽게 찾을 수 있을까?

- 소스 코드 (GUI 버전)

```
import tkinter as tk
from tkinter import ttk
from tkinter import scrolledtext

from hashlib import sha256 as sha
import time

#클릭 버튼 이벤트 처리 함수 작성
def click_start():

    tmp1 = message_entered.get()
    tmp2 = bits_entered.get()

    hashcash(tmp1, int(tmp2))

#해시캐시 함수 작성
def hashcash(msg, difficulty):
    nonce = 0

    stime=time.time()

    while True:
        target = '%s%d' %(msg, nonce)
        ret = sha(target.encode()).hexdigest()

        if ret[:difficulty] == '0'*difficulty:
            final_msg = msg + '\nGot it!! \n'

            etime = time.time()

            final_msg += f'HASH : {ret}\n'
            final_msg += f'NONCE : {nonce}'

            break

        nonce += 1

    t = etime-stime

    final_msg += f'\nrunning time : {t}\n\n'
    results_entered.insert(tk.INSERT, final_msg)

#GUI 첫 화면 생성 및 타이틀 작성
win = tk.Tk()
win.title("HASH CASH - PTU Infosec programming - 2022-4-30")
win.geometry("400x200")
#root.resizable(False, False)

#라벨 생성
ttk.Label(win, text="Message : ").grid(column=0, row=0)
```

```

#엔트리 위젯 (메시지 입력 텍스트 상자) 생성
message = tk.StringVar()
message_entered = ttk.Entry(win, width=22, textvariable=message)
message_entered.grid(column=1, row=0)

#라벨 생성
ttk.Label(win, text="Bits : ").grid(column=0, row=1, sticky=tk.E)

#엔트리 위젯 (메시지 입력 텍스트 상자) 생성
bits = tk.StringVar()
bits_entered = ttk.Entry(win, width=22, textvariable=bits)
bits_entered.grid(column=1, row=1)

#버튼 생성
b1 = ttk.Button(win, text="Start finding!!", command=click_start)
b1.grid(column=3, row=0)

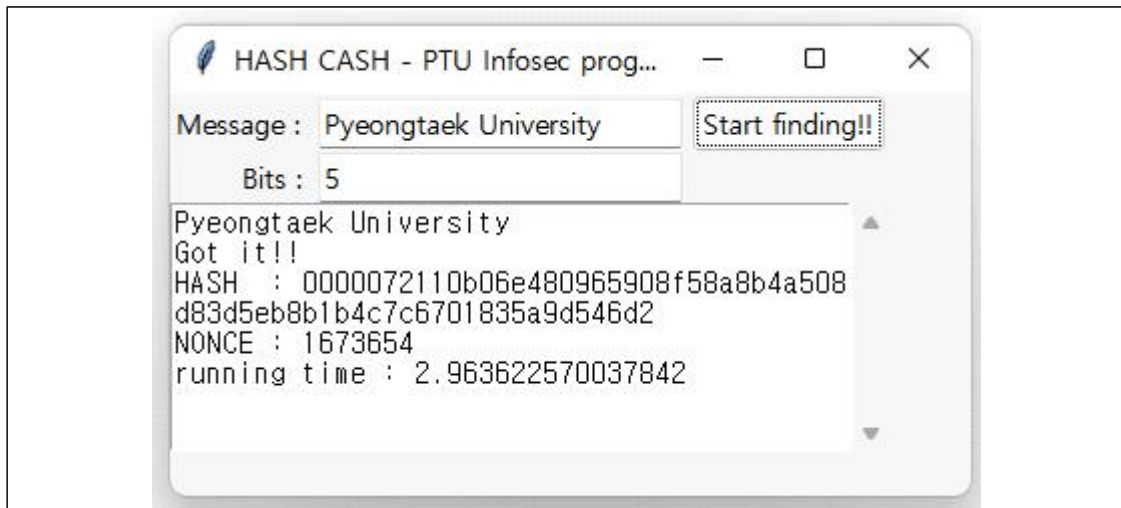
#스크롤 엔트리 위젯 (결과 출력 스크롤 텍스트 상자) 생성
results = tk.StringVar()
results_entered = scrolledtext.ScrolledText(win, width=42, height=8)
results_entered.grid(column=0, row=2, columnspan=4)

#첫 포커스 설정
message_entered.focus()

win.mainloop()

```

- 실행 결과



- 소스 코드 (console 버전)

```
from hashlib import sha256 as sha
import time

def hashcash(msg, difficulty):
    nonce = 0

    print('+++ Start')

    stime=time.time()

    while True:
        target = '%s%d' %(msg, nonce)

        #중간 target 출력 : 아래 라인 삭제 시 탐색 속도 50배 증가
        print('target : %s' %target)

        ret = sha(target.encode()).hexdigest()

        #중간 해시 출력 : 아래 라인 삭제 시 탐색 속도 50배 증가
        print('Hash : %s' %ret)

        if ret[:difficulty] == '0'*difficulty:
            print('+++ Bingo')

            etime = time.time()

            print('--->', ret)
            print('---> NONCE=%d' %nonce)
            break

        nonce += 1

    t = etime-stime
    print("running time : ", t)

def main():
    msg = 'Pyeongtaek University!'
    difficulty = 3

    hashcash(msg, difficulty)

main()
```

● 실행 결과

```
...
...
...
target : Pyeongtaek University!8646
Hash : cab927133dc3355b55b8fb1c1965ceec5bdec0113beb3cc62d35f48c5e880bc1
target : Pyeongtaek University!8647
Hash : 997b24e32f5dd003b48678b80bef8ae3458d2f06062148a5a2cf6592a0bad2fa
target : Pyeongtaek University!8648
Hash : 96eb666aeae2c1f5fece1fc2ae648de153e109a228da4424354ce0b8c5fbfc9f
target : Pyeongtaek University!8649
Hash : ac659de9482766642e0a76be365d7bc8efbe5366081481b63c224553ce5fb30d
target : Pyeongtaek University!8650
Hash : a75a9ba60a29b1e78bd8a62727d479a478b556aeba368d9c3ca7aa57600774ba
target : Pyeongtaek University!8651
Hash : 55e4e796e64d03a2a824433f46ac7d186e48bc9936f2d4e8a50d05ecb83d4db6
target : Pyeongtaek University!8652
Hash : dbb3c8fe99a9420f3eb0f5ecafbed58f155d9a6817d5aefebc2520544288faf4
target : Pyeongtaek University!8653
Hash : d6552747f8b397a6e5c88ca92d52def305c534d2fe52245d303b2cb50f1e9428
target : Pyeongtaek University!8654
Hash : 531f191ed826795a6adc7f9bcb7279e50c2cdc5c258ce6fba7a3c68c505d030a
target : Pyeongtaek University!8655
Hash : d5ebd1704a1af939af5422318c840a888a20c3dd4630770dac08e09f6abff581
target : Pyeongtaek University!8656
Hash : 00028e62333a32b5b6bdf41418f42725f8c21d84dc0b7ff60e4480cc11a8677b
+++ Bingo
---> 00028e62333a32b5b6bdf41418f42725f8c21d84dc0b7ff60e4480cc11a8677b
---> NONCE=8656
running time : 81.65891647338867
```

블록들의 POW solution 검증

2. 아래 코드는 125,552번째 블록에 대해 nonce값이 올바른 nonce값 임을 검증하는 코드이다. 다른 임의의 블록에 대해 올바른 nonce값이 맞는지 확인하는 코드를 작성하고 이에 대해 설명하라.

- 소스 코드 (콘솔 버전)

```
from hashlib import sha256 as sha
import codecs

# 인자 bits를 실제 16진수 값으로 리턴
def decodeBitcoinVal(bits):
    decode_hex = codecs.getdecoder('hex_codec')
    binn = decode_hex(bits)[0]
    ret = codecs.encode(binn[::-1], 'hex_codec')
    return ret

def getTarget(bits):
    bits = decodeBitcoinVal(bits)
    bits = int(bits, 16)
    bit1 = bits >> 4 * 6
    base = bits & 0x00ffffff

    sft = (bit1 - 0x3) * 8
    target = base << sft

    return target

# 아래 에는 2011년 5월에 생성된 비트코인 125,552번째 블록에 대한 작업증명이
# 성공했음을 검증하는 코드임
def validatePoW(header):
    block_version = header[0]
    hashPrevBlock = header[1]
    hashMerkleRoot = header[2]
    Time = header[3]
    Bits = header[4]
    nonce = header[5]

    ts_str = str(hex(Time))[2:]

    # 블록 헤더의 모든 값을 더하고 이를 바이트 객체로 변경
    decode_hex = codecs.getdecoder('hex_codec')

    # 문자열로 변환!!
    header_hex = block_version + decodeBitcoinVal(hashPrevBlock).decode() +
    decodeBitcoinVal(hashMerkleRoot).decode() + decodeBitcoinVal(ts_str).decode() +
    Bits + nonce
    header_bin = decode_hex(header_hex)[0]

    hash = sha(header_bin).digest()
    hash = sha(hash).digest()
    PoW = codecs.encode(hash[::-1], 'hex_codec')

    # 헤더의 Bits 값을 이용해 실제 target 값 추출

    target = getTarget(Bits)
    target = str(hex(target))
    target = '0' * (66 - len(target)) + target[2:]

    print('target\t=', target)
    print('PoW\t=', PoW.decode())

    # 작업 증명이 성공한 건지 체크
    if int(PoW, 16) <= int(target, 16):
        print('+++ Accept this Block')
    else:
        print('--- Reject this Block')
```


● 소스 코드 (콘솔 버전)

```
from hashlib import sha256 as sha
import codecs

# 인자 bits를 실제 16진수 값으로 리턴
def decodeBitcoinVal(bits):
    decode_hex = codecs.getdecoder('hex_codec')
    binn = decode_hex(bits)[0]
    ret = codecs.encode(binn[::-1], 'hex_codec')
    return ret

def getTarget(bits):
    bits = decodeBitcoinVal(bits)
    bits = int(bits, 16)
    bit1 = bits >> 4 * 6
    base = bits & 0x00ffffff

    sft = (bit1 - 0x3) * 8
    target = base << sft

    return target

# 아래 에는 2011년 5월에 생성된 비트코인 125,552번째 블록에 대한 작업증명이
# 성공했음을 검증하는 코드임
def validatePoW(header):
    block_version = header[0]
    hashPrevBlock = header[1]
    hashMerkleRoot = header[2]
    Time = header[3]
    Bits = header[4]
    nonce = header[5]

    # 블록 헤더의 모든 값을 더하고 이를 바이트 객체로 변경
    decode_hex = codecs.getdecoder('hex_codec')

    # 문자열로 변환!!
    header_hex = block_version + decodeBitcoinVal(hashPrevBlock).decode() +
    decodeBitcoinVal(hashMerkleRoot).decode() + Time + Bits + nonce
    header_bin = decode_hex(header_hex)[0]

    hash = sha(header_bin).digest()
    hash = sha(hash).digest()
    PoW = codecs.encode(hash[::-1], 'hex_codec')

    # 헤더의 Bits 값을 이용해 실제 target 값 추출

    target = getTarget(Bits)
    target = str(hex(target))
    target = '0' * (66 - len(target)) + target[2:]

    print('target\t=', target)
    print('PoW\t=', PoW.decode())

    # 작업 증명이 성공한 건지 체크
    if int(PoW, 16) <= int(target, 16):
        print('+++ Accept this Block')
    else:
        print('--- Reject this Block')

block_version = '02000000'
hashPrevBlock =
'000000000000000067ecc744b5ae34eebbde14d21ca4db51652e4d67e155f07e'
hashMerkleRoot =
'915c887a2d9ec3f566a648bedcf4ed30d0988e22268cfe43ab5b0cf8638999d3'

# 입력을 정수값으로 받음!
Time = '02c86d53'
Bits = '6c890019'
nonce = '593a470d'
```

```
header = [block_version, hashPrevBlock, hashMerkleRoot, Time, Bits, nonce]
validatePoW(header)
```

- 실행 결과

```
target  = 00000000000000000896c0000000000000000000000000000000000000000000
PoW     = 0000000000000000082ccf8f1557c5d40b21edabb18d2d691cfbf87118bac7254
+++ Accept this Block
```


Notes and comments

1. 2022년 4월 30일 작성

- (1) GUI버전에서 현재 찾고 있는 nonce 값을 화면에 실시간으로 보여주도록 하면 기다리는 것이 좀 덜 지루하지 않을까? 화면에 보여주는 것의 리소스를 덜 소모하도록 독립적인 text 위젯을 만들도록 하자!
- (2) Java로도 구현해 보자!
- (3) 콘솔 버전의 코드는, 교재 (화이트 해커를 위한 암호화 해킹, 정보문화사, 장삼용 지음)의 코드를 바탕으로 작성되었음.

2. 2022년 5월 08일 작성

- (4) PoW solution 확인 코드는 교재 (화이트 해커를 위한 암호화 해킹, 정보문화사, 장삼용 지음)의 코드를 바탕으로 작성되었음.

3. 2023년 5월 17일 작성

- (5) 검증 시 Time 값, Bits 값, nonce 값이 리틀엔디안 형식으로 인해 변경되어 관련된 코드 일부 변형되었음.

4. 2025년 5월 27일 작성

- (6) validation 코드 : 값 변경 부분을 통일시킬 순 없을까?

5. 2026년 5월 13일 작성

- (7) validation 코드 : 시간 값을 나머지 요소와 동일하게 반대로 입력하도록 재작성